ML Experiment-06

In [1]:
```
import random
import numpy as np
```

In [2]:
```
class TicTacToe:
    def __init__(self):
        self.reset()

    def reset(self):
        self.board = [0] * 9
        self.done = False
        return self.get_state()

    def get_state(self):
        return tuple(self.board)

    def available_actions(self):
        return [i for i, value in enumerate(self.board) if value == 0]

    def check_winner(self):
        winning_lines = [
            (0,1,2), (3,4,5), (6,7,8),
            (0,3,6), (1,4,7), (2,5,8),
            (0,4,8), (2,4,6)
        ]

        for a, b, c in winning_lines:
            s = self.board[a] + self.board[b] + self.board[c]

            if s == 3:
                return 1
            if s == -3:
                return -1

        if 0 not in self.board:
            return 0

        return None

    def step(self, action, player=1):
        if self.done or action not in self.available_actions():
            return self.get_state(), -10, True

        self.board[action] = player
        result = self.check_winner()

        if result is not None:
            self.done = True

            if result == player:
                return self.get_state(), 1, True
            elif result == 0:
                return self.get_state(), 0, True
```

```
            else:
                return self.get_state(), -1, True

        return self.get_state(), 0, False

    def render(self):
        symbols = {1: "X", -1: "O", 0: " "}

        for r in range(3):
            print(
                f" {symbols[self.board[3*r]]} |"
                f" {symbols[self.board[3*r+1]]} |"
                f" {symbols[self.board[3*r+2]]} "
            )

            if r < 2:
                print("---+---+---")


class QLearningAgent:
    def __init__(
        self,
        alpha=0.1,
        gamma=0.9,
        epsilon=1.0,
        epsilon_decay=0.995,
        epsilon_min=0.05
    ):
        self.q_table = {}
        self.alpha = alpha
        self.gamma = gamma
        self.epsilon = epsilon
        self.epsilon_decay = epsilon_decay
        self.epsilon_min = epsilon_min

    def get_q_values(self, state):
        if state not in self.q_table:
            self.q_table[state] = np.zeros(9)

        return self.q_table[state]

    def choose_action(self, state, available_actions, training=True):
        q_values = self.get_q_values(state)

        if training and random.random() < self.epsilon:
            return random.choice(available_actions)

        legal_q = [(a, q_values[a]) for a in available_actions]
        max_q = max(value for _, value in legal_q)
        best_actions = [
            a for a, value in legal_q if value == max_q
        ]

        return random.choice(best_actions)

    def update(self, state, action, reward, next_state, done):
```

```
        current_q = self.get_q_values(state)[action]

        if done:
            target = reward
        else:
            next_q = self.get_q_values(next_state)
            target = reward + self.gamma * np.max(next_q)

        self.q_table[state][action] += self.alpha * (target - current_q)

    def decay_epsilon(self):
        self.epsilon = max(
            self.epsilon_min,
            self.epsilon * self.epsilon_decay
        )


def train_agent(episodes=50000):
    agent = QLearningAgent()

    for episode in range(episodes):
        env = TicTacToe()
        state = env.reset()

        while not env.done:
            actions = env.available_actions()
            action = agent.choose_action(
                state, actions, training=True
            )

            next_state, reward, done = env.step(action, player=1)

            agent.update(
                state, action, reward, next_state, done
            )

            state = next_state

            if done:
                break

            opponent_actions = env.available_actions()

            if opponent_actions:
                opponent_action = random.choice(opponent_actions)

                next_state, opponent_reward, done = env.step(
                    opponent_action, player=-1
                )

                if done and opponent_reward == 1:
                    opponent_reward = -1

                agent.update(
                    state, action, opponent_reward,
                    next_state, done
```

```
                )

                state = next_state

        agent.decay_epsilon()

    return agent


def test_agent(agent, games=10):
    wins = draws = losses = 0

    for game in range(games):
        env = TicTacToe()
        state = env.reset()

        print(f"\nTest Game {game + 1}")
        env.render()

        while not env.done:
            # Agent plays X
            actions = env.available_actions()
            action = agent.choose_action(
                state, actions, training=False
            )

            state, reward, done = env.step(action, player=1)
            print("\n")
            env.render()

            if done:
                break

            actions = env.available_actions()
            opponent_action = random.choice(actions)

            state, opponent_reward, done = env.step(
                opponent_action, player=-1
            )
            print("\n")
            env.render()

        result = env.check_winner()

        if result == 1:
            wins += 1
            print("Result: Agent Wins")
        elif result == -1:
            losses += 1
            print("Result: Agent Loses")
        else:
            draws += 1
            print("Result: Draw")

    print("\nTesting Summary")
    print("Wins  :", wins)
```

```
    print("Draws :", draws)
    print("Losses:", losses)

agent = train_agent(episodes=50000)

print("Training completed.")
print("Number of learned states:", len(agent.q_table))
print("Final epsilon:", agent.epsilon)

test_agent(agent, games=10)
```

```
Training completed.
Number of learned states: 3400
Final epsilon: 0.05

Test Game 1
   |   |
---+---+---
   |   |
---+---+---
   |   |


 X |   |
---+---+---
   |   |
---+---+---
   |   |


 X |   |
---+---+---
   | O |
---+---+---
   |   |


 X |   | X
---+---+---
   | O |
---+---+---
   |   |


 X |   | X
---+---+---
   | O |
---+---+---
 O |   |


 X | X | X
---+---+---
   | O |
---+---+---
 O |   |
Result: Agent Wins

Test Game 2
   |   |
---+---+---
   |   |
---+---+---
   |   |


 X |   |
```

```
   |   |
---+---+---
   |   |
---+---+---
   |   |


 X |   |
---+---+---
 O |   |
---+---+---
   |   |


 X |   | X
---+---+---
 O |   |
---+---+---
   |   |


 X |   | X
---+---+---
 O |   |
---+---+---
   | O |


 X | X | X
---+---+---
 O |   |
---+---+---
   | O |
Result: Agent Wins

Test Game 3
   |   |
---+---+---
   |   |
---+---+---
   |   |


 X |   |
---+---+---
   |   |
---+---+---
   |   |


 X |   |
---+---+---
   |   | O
---+---+---
   |   |
```

```
 X |   |
---+---+---
   |   | O
---+---+---
   |   | X


 X |   | O
---+---+---
   |   | O
---+---+---
   |   | X


 X |   | O
---+---+---
   | X | O
---+---+---
   |   | X
Result: Agent Wins

Test Game 4
   |   |
---+---+---
   |   |
---+---+---
   |   |


 X |   |
---+---+---
   |   |
---+---+---
   |   |


 X | O |
---+---+---
   |   |
---+---+---
   |   |


 X | O | X
---+---+---
   |   |
---+---+---
   |   |


 X | O | X
---+---+---
   |   |
---+---+---
   |   | O
```

```
 X | O | X
---+---+---
 X |   |
---+---+---
   |   | O


 X | O | X
---+---+---
 X |   | O
---+---+---
   |   | O


 X | O | X
---+---+---
 X |   | O
---+---+---
 X |   | O
Result: Agent Wins

Test Game 5
   |   |
---+---+---
   |   |
---+---+---
   |   |


 X |   |
---+---+---
   |   |
---+---+---
   |   |


 X |   |
---+---+---
   |   |
---+---+---
   |   | O


 X |   |
---+---+---
   |   |
---+---+---
 X |   | O


 X |   |
---+---+---
   |   | O
---+---+---
 X |   | O
```

```
 X |   |
---+---+---
 X |   | O
---+---+---
 X |   | O
Result: Agent Wins

Test Game 6
   |   |
---+---+---
   |   |
---+---+---
   |   |


 X |   |
---+---+---
   |   |
---+---+---
   |   |

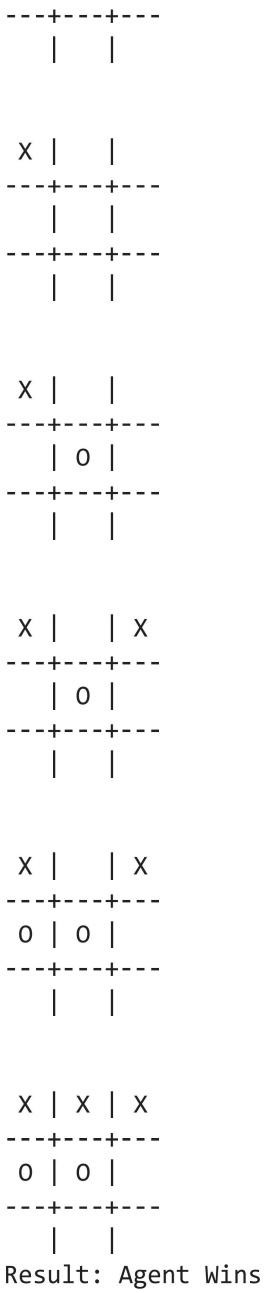

 X |   |
---+---+---
   |   |
---+---+---
   |   | O


 X |   |
---+---+---
   |   |
---+---+---
 X |   | O


 X |   |
---+---+---
   | O |
---+---+---
 X |   | O

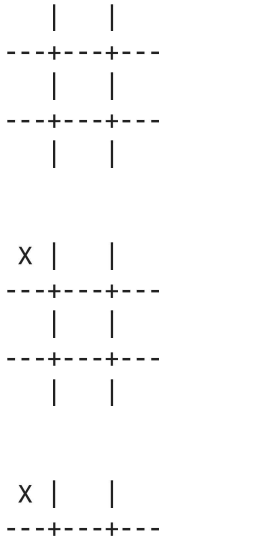

 X |   |
---+---+---
 X | O |
---+---+---
 X |   | O
Result: Agent Wins

Test Game 7
   |   |
---+---+---
   |   |
```



```
Result: Agent Wins

Test Game 8
```



```
   |   |
---+---+---
   | O |


 X |   |
---+---+---
   |   |
---+---+---
 X | O |


 X |   |
---+---+---
   |   |
---+---+---
 X | O | O


 X |   |
---+---+---
 X |   |
---+---+---
 X | O | O
Result: Agent Wins

Test Game 9
   |   |
---+---+---
   |   |
---+---+---
   |   |


 X |   |
---+---+---
   |   |
---+---+---
   |   |


 X | O |
---+---+---
   |   |
---+---+---
   |   |


 X | O | X
---+---+---
   |   |
---+---+---
   |   |


 X | O | X
```

```
---+---+---
   | O |
---+---+---
   |   |


 X | O | X
---+---+---
   | O |
---+---+---
 X |   |


 X | O | X
---+---+---
   | O |
---+---+---
 X | O |
Result: Agent Loses

Test Game 10
   |   |
---+---+---
   |   |
---+---+---
   |   |


 X |   |
---+---+---
   |   |
---+---+---
   |   |


 X |   |
---+---+---
   | O |
---+---+---
   |   |


 X |   | X
---+---+---
   | O |
---+---+---
   |   |


 X |   | X
---+---+---
 O | O |
---+---+---
   |   |
```

```
 X | X | X
---+---+---
 O | O |
---+---+---
   |   |
Result: Agent Wins

Testing Summary
Wins  : 9
Draws : 0
Losses: 1
```